
Tumor 3D Reconstruction

Project for the class of Computer Vision, 2026

Simon KHAN^{*1} Feddy IMMOULA^{*1} Thibault POUX¹

Abstract

This report presents a method for 3D tumor reconstruction from MRI scans. We leverage a 2D U-Net architecture for slice-by-slice segmentation, followed by a robust 3D volumetric post-processing pipeline. The method is evaluated on a medical dataset, demonstrating the effectiveness of combining deep learning with morphological constraints for accurate volumetric estimation. [GitHub](#)

1. Introduction

1.1. Medical Context

Liver cancer is a leading cause of global cancer mortality, requiring precise tumor localization for effective treatment. In clinical workflows, Computed Tomography (CT) is the primary modality for liver assessment. Accurate segmentation of hepatic lesions is critical for **surgical planning**, where 3D spatial relationships dictate resectability, and for **oncological monitoring** via RECIST criteria. However, manual 3D segmentation is time-consuming and prone to inter-observer variability. Automated tools are therefore essential to provide reproducible, high-fidelity 3D reconstructions that assist in complex clinical decision-making and improve patient outcomes.

1.2. State of the Art

The field has shifted from classical active contours to Deep Learning, with the **U-Net** architecture becoming the standard for medical segmentation. State-of-the-art methods from the LiTS benchmark typically utilize:

- **2D CNNs:** Efficient architectures that leverage *Transfer Learning* from pre-trained ImageNet backbones to compensate for limited medical data.
- **3D CNNs: (?)** Models capturing volumetric context at the cost of high memory consumption and complexity.
- **Hybrid Models:**([Li et al., 2018](#)) Networks like H-DenseUNet that combine 2D intra-slice and 3D inter-slice features.

Current research focuses on addressing **class imbalance** using Dice or Focal losses and refining outputs through volumetric post-processing. Our approach utilizes a 2D U-Net with a pre-trained backbone, prioritized for its balance between computational efficiency and segmentation accuracy.

1.3. Dataset Description

We use a dataset that come from the LITS challenge ([Bilic et al., 2023](#)). The LiTS dataset is a benchmark designed to provide a gold standard for the development and evaluation of algorithms that can automatically segment the liver and its tumors from abdominal CT scans. It was created to address the significant challenge of high variability in liver shapes and the diverse appearances of tumors in clinical imaging.

1.3.1. COMPOSITION

The dataset consists of a large collection of CT volumes that are categorized into different levels of annotation:

- **Imaging Data:** These are the raw 3D CT scans of the abdomen, which include various clinical cases with different tumor types (e.g., hepatocellular carcinoma, metastasis).
- **Liver Mask:** This layer provides the ground truth segmentation for the entire liver volume.
- **Tumor Mask:** This layer provides specific segmentations for the lesions or tumors found within the liver.

The data was collected from several clinical institutions worldwide, ensuring a wide variety of scanners and protocols are represented

1.3.2. OUR USE CASE

The dataset was previously featured in a CodaLab challenge titled "LiTS - Liver Tumor Segmentation Challenge." The primary objective of this challenge was to encourage researchers to develop automatic algorithms capable of segmenting liver lesions in contrast-enhanced abdominal CT scans.

We selected this dataset because it is large enough to effectively train a robust enough model while remaining small enough to accommodate our limited computational resources. The dataset consists of 131 different 3D CT scans. For our implementation, we divided these scans into three distinct parts: a training set, a test set, and a validation set, using a split ratio of 0.80 for training, 0.15 for testing, and 0.05 for validation.

2. Methods

This section details our two-stage pipeline: initial 2D segmentation followed by 3D refinement.

2.1. 2D U-Net Segmentation

The first stage of the pipeline performs tumor segmentation on contrast-enhanced abdominal **CT** scans using a 2D U-Net. Instead of training on full axial slices, we adopt a **patch-based learning strategy** to reduce GPU memory and I/O pressure and, more importantly, to mitigate the severe class imbalance (tumor pixels are rare and many slices contain no lesion). Patch-based learning increases the frequency of informative samples and allows stable training under limited computational resources.

2.1.1. U-NET MODEL

We implement a standard 2D U-Net encoder-decoder (based on [Pytorch-UNet \(GitHub\)](#)) with skip connections, following the original “contracting path / expanding path” design. The model takes as input a single-channel CT patch and outputs a single-channel *logit* map of the same spatial size. A sigmoid is applied at inference time to obtain tumor probabilities.

Building blocks. The network is built from four main modules defined in `unet_parts.py`:

- **DoubleConv:** two consecutive 3×3 convolutions (padding = 1) each followed by Batch Normalization and ReLU. An optional Dropout can be inserted after activations when `dropout > 0`. This block is used everywhere in the encoder and decoder.
- **Down:** a 2×2 MaxPool (stride 2) followed by a DoubleConv. This halves spatial resolution while increasing feature channels.
- **Up:** an upsampling by a factor 2 followed by concatenation with the corresponding encoder feature map (skip connection), then a DoubleConv to fuse features. In our configuration we use **bilinear upsampling** (`mode='bilinear', align_corners=True`) to reduce checkerboard artifacts and memory usage. When bilinear upsampling

is used, the number of channels at the bottleneck is adjusted by a factor of 2.

- **OutConv:** a final 1×1 convolution that maps the last decoder feature map to the output channel (binary tumor logit).

Encoder-decoder configuration. Starting from 64 channels after the first DoubleConv, the encoder increases feature dimensionality as follows:

$$64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024,$$

where each transition is performed by a Down block. When `bilinear=True`, we set a channel *factor* of 2 at the bottleneck so that the deepest block uses $1024/2 = 512$ channels, reducing memory and computation. The decoder mirrors the encoder using four Up blocks to progressively recover the original spatial resolution and fuse high-level semantics with low-level details through skip connections.

Shape alignment inside the Up block. Because pooling/upsampling and integer divisions can introduce small size mismatches, the Up module explicitly aligns feature map shapes before concatenation. After upsampling, the decoder feature map is padded using `torch.nn.functional.pad` so that its height/width match the encoder skip tensor, then concatenated along channels (`torch.cat(..., dim=1)`). This guarantees a valid skip fusion even when dimensions are not perfectly divisible by 2^k .

Memory-saving options. Two implementation choices were used to fit GPU constraints:

- **Bilinear upsampling** instead of transposed convolutions (lower memory footprint, fewer artifacts).
- **Gradient checkpointing (optional).** The model exposes a `use_checkpointing()` method that wraps blocks with `torch.utils.checkpoint`. This trades compute for memory by recomputing intermediate activations during backpropagation, which can be helpful on constrained GPUs.

2.1.2. PATCH-BASED DATASET CONSTRUCTION

Motivation. Training on full $H \times W$ slices is inefficient in our setting: (i) background dominates, (ii) many slices contain no tumor, and (iii) full-slice training increases both GPU memory usage and disk I/O. Using fixed-size patches yields more diverse samples, increases the effective proportion of positive examples, and allows larger batch sizes.

Slice decomposition and binary tumor mask. Each 3D CT volume is decomposed into axial 2D slices. The corresponding ground truth segmentation is converted into a binary tumor mask. In LiTS annotations, tumor corresponds to label value 2, therefore tumor pixels are detected with (`seg == 2`). This binary mask is used to determine whether a candidate patch contains tumor pixels and to compute patch-level supervision.

Patch extraction with padding. Given a patch size P (in our experiments $P = 256$), a square crop is extracted from each slice. To make patch extraction robust near image borders, we apply symmetric padding before cropping. With $p = \lfloor P/2 \rfloor$, the slice and mask are padded in the spatial dimensions so that any selected center (x, y) yields a valid $P \times P$ crop. This is implemented with `np.pad` by specifying per-dimension padding widths (no padding on the slice index / channel dimension, padding of p pixels on both sides of height and width). After padding, we extract a centered crop and return it as one training sample.

Intensity normalization (HU windowing). Each CT patch is normalized using Hounsfield Unit windowing:

$$I' = \frac{\text{clip}(I, l, h) - l}{h - l}, \quad (1)$$

with $l = -100$ and $h = 400$, mapping intensities to $[0, 1]$. This improves contrast in the soft tissue range and stabilizes optimization.

2.1.3. CLASS IMBALANCE HANDLING

Tumor segmentation is highly imbalanced both *within patches* (tumor covers a small fraction of pixels) and *across samples* (many patches may contain no tumor). We mitigate this imbalance with complementary mechanisms:

Pixel-level reweighting (BCE `pos.weight`). We use `BCEWithLogitsLoss` with a positive class weight (`pos.weight`) to penalize false negatives more strongly than false positives. This encourages sensitivity to tumor pixels despite their sparsity and reduces the tendency to predict all-background masks.

Foreground-focused overlap learning (Dice). We also optimize a Dice-based term which directly targets region overlap. In practice, overlap-based gradients can be unstable on empty targets, therefore we compute a **foreground Dice** contribution when relevant (i.e., focusing the overlap signal on samples containing tumor), which improves learning of tumor shape and limits degenerate solutions.

2.1.4. LOSS FUNCTION AND EVALUATION METRIC

The training objective combines a pixel-wise classification term and a region-overlap term:

$$\mathcal{L} = \mathcal{L}_{Dice} + \mathcal{L}_{BCE}. \quad (2)$$

The model outputs logits, and the BCE term is applied directly on logits for numerical stability. Dice is computed on sigmoid probabilities.

For validation, we report a Dice score on the tumor class (foreground). This metric is better aligned with segmentation quality than raw loss values, especially in strongly imbalanced settings.

2.1.5. 2D U-NET RESULTS OVER 100 EPOCHS

To analyze training dynamics, we monitor the training loss, validation loss, and validation Dice score across epochs (Figure 1). These curves highlight two key phenomena: a consistent optimization on the training set and a much noisier behavior on the validation set.

Interpretation of the curves. Figure 1(a) shows that the training loss decreases steadily, indicating that the network fits the training distribution effectively. However, Figure 1(b) shows that the validation loss increases after the early epochs and remains noisy, which is a typical sign of **overfitting** (or, more generally, a distribution mismatch between train and validation patches).

At the same time, Figure 1(c) shows that validation Dice improves quickly in the first epochs, then oscillates strongly without a stable upward trend. This behavior is common in patch-based tumor segmentation because (i) the validation set is usually small (few volumes and few positive regions), (ii) tumor appearance varies a lot across patients, and (iii) Dice is extremely sensitive to small boundary changes when tumors are small.

Loss vs Dice: which one to trust? In imbalanced segmentation, the Dice score is generally more informative for clinical segmentation quality because it directly measures overlap. The validation loss is still useful as an optimization signal but can be harder to interpret: it mixes calibration effects (BCE) and overlap effects (Dice), and can increase even when Dice stays acceptable due to confidence miscalibration or class weighting. For model selection, we therefore prioritize the **best validation Dice checkpoint** rather than the minimum validation loss.

Practical mitigation strategies. To reduce this noisy validation behavior and improve generalization, we rely on: early stopping based on validation Dice, stronger regularization (weight decay, optional dropout in deeper blocks).

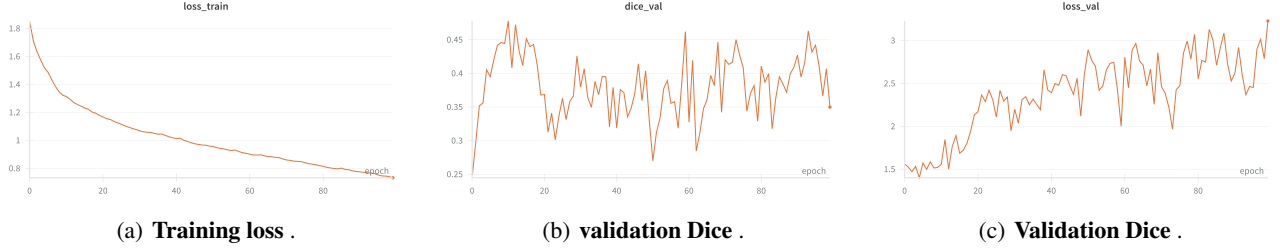


Figure 1. 2D U-Net training curves on the patch-based dataset. The training loss decreases smoothly, while the validation loss increases and becomes highly variable. The validation Dice is also noisy, indicating high variance in generalization performance on the validation split.

2.1.6. TRAINING PROCEDURE AND COMPUTATIONAL CONSTRAINTS

Training is performed on GPU using mixed precision (AMP) when available (`autocast + GradScaler`) to reduce memory usage and speed up training. We use the AdamW optimizer with weight decay regularization. We also implement checkpointing by saving both the last model and the best-performing model according to validation Dice, enabling robust continuation and early stopping.

Due to Google Colab constraints (limited session time, limited GPU memory, and disk I/O overhead when loading NIfTI volumes), patch-based training is crucial to keep iterations feasible. Smaller samples reduce per-iteration transfer and allow more iterations per unit time.

One of our primary constraints was the computational limit of our hardware, as we do not own a dedicated GPU for training. Consequently, we relied on online resources to train our model. Initially, we used a Google Colab L4 GPU, but training was very slow. However, after implementing several Colab-specific optimizations, we were able to achieve significantly better performance. These optimizations included:

- **Data Transfer:** Moving the data from Google Drive directly onto the GPU host machine to minimize latency between mini-batch loadings. This drastically reduced the overall epoch time. These optimizations are crucial because data stored in Google Drive may reside in a different regional datacenter than the GPU host, and no automatic synchronization is implemented.
- **Compression:** Transferring the data in a compressed format to minimize transfer time. This reduced the setup time from 30 minutes to only 10 minutes.
- **WandB Logging:** Utilizing the Weights & Biases (WandB) logger to log our data locally using a separate thread. This prevents the training process from idling while waiting for network latency when sending data to the WandB servers.

Later, we were able to switch to an NVIDIA A100 GPU to further accelerate the learning process. The drawback, however, was that this consumed our Colab credits much faster, giving us approximately only 8 hours of total compute time on the A100.

Overall, through these optimizations, we managed to reduce the training time per epoch from 15 minutes on an L4 GPU to just 1 minute and 30 seconds. On the A100 GPU, this was further reduced to only 25 seconds per epoch. This efficiency allowed us to expand the scope of our project and perform a comprehensive hyperparameter optimization.

2.1.7. HYPERPARAMETERS OPTIMISATION

To improve the performance of our models, we implemented a hyperparameter optimization task using **Optuna**.

We selected the **TPE (Tree-structured Parzen Estimator)** sampler because it is more efficient than a basic grid search. This approach allows the optimization process to move directly toward configurations that improve the model by effectively balancing exploration and exploitation a necessity given our computational constraints.

Due to our limited budget, we carefully selected a specific subset of hyperparameters to optimize. Our search space was defined as follows:

Table 1. Hyperparameter Search Space

Parameter	Distribution	Range / Values
Dropout	Float	0.0 – 0.3 (step 0.1)
Learning Rate	Log-Uniform	10^{-5} – 10^{-3}
Weight Decay	Log-Uniform	10^{-6} – 10^{-3}
Bilinear	Categorical	{True, False}

We also implemented **early stopping** with a patience of 10 and a `min_delta` of 10^{-3} . This allowed us to terminate the training process once the improvements in the Dice score no longer justified the additional computational expense. Furthermore, we utilized **Optuna’s pruning** feature, which was configured to trigger after 3 epochs. This enabled us

to quickly eliminate underperforming hyperparameter sets, saving significant time and resources.

We performed 40 different trials, logging all metrics to Weights & Biases (WandB) to monitor the results. An overview of these results is provided below:

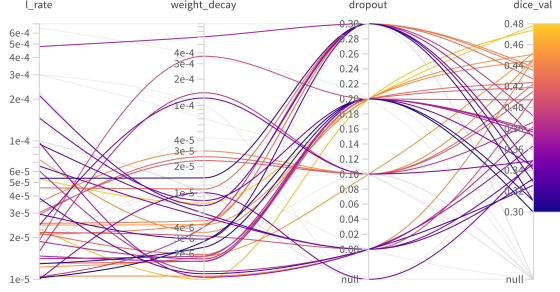


Figure 2. Hyperparameter optimization results in terms of Dice score

Several points regarding this graph 2 are noteworthy. First, information regarding the Bilinear hyperparameter is missing, due to an indentation error in the Python code, trials with `Bilinear=True` repeatedly crashed. By the time the issue was identified, it was too late in the Optuna study to restart those specific trials.

Additionally, the Learning Rate and Weight Decay columns are displayed on a logarithmic scale. The visualization clearly demonstrates that these three hyperparameters significantly influence model performance, with Dice scores ranging from approximately 0.30 to 0.55. It should be noted that the scores shown in this graph represent the Dice score of the final epoch for each specific trial. However, our “true” reported Dice score is slightly different, as we evaluated model performance based on the **best** Dice score achieved across all epochs, rather than just the final one.

Consequently, our best-performing model achieved a **Dice score of 0.5495** with the following parameters:

Table 2. Best Hyperparameter Configuration Found via Optuna

Hyperparameter	Optimal Value
Bilinear	False
Dropout	0.2
Learning Rate	2.315×10^{-5}
Weight Decay	1.018×10^{-6}

By utilizing Early Stopping, the training for the best trial concluded at epoch 32, effectively preventing overfitting and conserving computational resources. Through this optimization process, we successfully increased our Dice score from

an initial 0.452 to 0.5495.

While it would have been beneficial to optimize additional factors such as alternative activation functions (e.g., GELU or Leaky ReLU) or the batch size, these were excluded from the search space to remain within our strict computational constraints.

2.2. 3D Volumetric Post-processing

While the 2D U-Net architecture efficiently captures intra-slice features, it processes each axial slice independently. This slice-by-slice approach often leads to spatial inconsistencies along the Z-axis, such as “flickering” predictions or isolated false positives that lack volumetric coherence. To reconstruct a medically plausible 3D tumor mask, we implemented a robust post-processing pipeline. This pipeline aggregates the 2D probability maps into a 3D tensor and applies a sequence of regularization, binarization, and morphological filtering operations.

2.2.1. VOLUME AGGREGATION AND ALIGNMENT

The initial output of our segmentation network is a collection of 2D probability maps, $P_{2D} \in [0, 1]^{H \times W}$, generated for each slice of the input volume. We first stack these maps to form a raw 3D probability volume $V_{raw} \in [0, 1]^{Z \times H \times W}$.

A critical step in medical imaging is ensuring the prediction aligns with the physical coordinate system of the patient. We perform a shape alignment transformation T to map the dimensions of V_{raw} (typically stacked along the Z-axis) to the reference NIfTI geometry (X, Y, Z) . This ensures that subsequent volumetric operations respect the anisotropic voxel spacing often found in MRI data.

2.2.2. PROBABILISTIC REGULARIZATION

Before making a binary decision, we apply Gaussian smoothing to the probability volume. This step serves as a regularization prior, assuming that biological structures like tumors are continuous in space.

Our method allows for two smoothing strategies, defined by a configuration $\mathcal{S}$:

- **Isotropic 3D Smoothing:** We convolve the volume with a 3D Gaussian kernel G_σ . This suppresses high-frequency noise in all directions but may blur boundaries if σ is too large.
- **Z-axis Anisotropic Smoothing:** Given that the U-Net already learns strong correlations in the XY plane, the lack of continuity is most prominent along the Z-axis. We effectively apply a 1D Gaussian filter along the depth dimension:

$$V_{smooth}(x, y, z) = (V_{raw}(x, y, \cdot) * g_{\sigma_z})(z) \quad (3)$$

where g_{σ_z} is a 1D Gaussian kernel. This specifically targets inter-slice discontinuities without altering the sharp boundaries detected by the U-Net in the axial plane.

2.2.3. HYSTERESIS THRESHOLDING

To convert the continuous probability maps V_{smooth} into a binary mask M_{bin} , simple global thresholding (e.g., $t = 0.5$) can be insufficient, often resulting in fragmented masks where the tumor probability dips slightly below the threshold.

To address this, we implemented **Hysteresis Thresholding**. This technique utilizes two thresholds, t_{low} and t_{high} (where $t_{low} < t_{high}$).

The binary mask is constructed in two steps:

1. **Core detection:** We identify "strong" tumor voxels where $P(v) > t_{high}$.
2. **Region growing:** We expand these regions to include adjacent "weak" voxels where $P(v) > t_{low}$, recursively connected to the strong core.

This ensures that lower-confidence regions are only preserved if they are structurally connected to high-confidence tumor tissue, effectively reducing isolated noise while preserving tumor margins.

2.2.4. CONNECTED COMPONENT ANALYSIS (CCA)

The final stage of our pipeline acts as a morphological filter to remove false positives based on geometric constraints. We treat the binary mask as a graph where voxels are nodes and adjacency is defined by 26-connectivity (including diagonals).

We perform Connected Component Labeling (CCL) to decompose the mask into K disjoint sets $\{C_1, C_2, \dots, C_K\}$. For each component C_k , we compute its physical volume $Vol(C_k)$ in cubic centimeters (cm^3) using the voxel spacing $(\delta_x, \delta_y, \delta_z)$ derived from the NIfTI header:

$$Vol(C_k) = |C_k| \times (\delta_x \cdot \delta_y \cdot \delta_z) \times 10^{-3} \quad (4)$$

Our filtering strategy combines two criteria:

1. **Minimum Volume Thresholding:** We discard any component where $Vol(C_k) < \tau_{vol}$ (e.g., $0.5cm^3$). This effectively removes small, isolated artifacts that are biologically unlikely to be significant tumors.
2. **Top-N Selection:** In scenarios assuming a single primary tumor, we rank the remaining components by size and retain only the largest one (or Top- N).

Algorithm 1 3D Reconstruction Pipeline

Input: Stack of 2D Proba Maps $\mathbf{P}_{stack}$
Parameters: σ (smooth), $t_{low}, t_{high}, V_{min}$
 Align $\mathbf{V} \leftarrow \text{Transpose}(\mathbf{P}_{stack})$ to Reference NIfTI
Step 1: Smoothing
 $\mathbf{V} \leftarrow \mathbf{V} * G_{\sigma}$ {Gaussian convolution}
Step 2: Hysteresis Thresholding
 $\mathbf{M}_{strong} \leftarrow (\mathbf{V} > t_{high})$
 $\mathbf{M}_{weak} \leftarrow (\mathbf{V} > t_{low})$
 $\mathbf{M}_{bin} \leftarrow \text{Reconstruct}(\mathbf{M}_{weak}, \text{seed} = \mathbf{M}_{strong})$
Step 3: CCA Filtering
 Labels $\mathcal{L} \leftarrow \text{Label}(\mathbf{M}_{bin}, \text{connectivity} = 26)$
for each component C_i in $\mathcal{L}$ **do**
 if PhysicalVolume(C_i) $< V_{min}$ **then**
 $\mathbf{M}_{bin}[C_i] \leftarrow 0$ {Remove noise}
 end if
end for
 Keep largest remaining component(s)
Return: Final 3D Mask $\mathbf{M}_{bin}$

The complete reconstruction algorithm is summarized in Algorithm 1.

2.2.5. SURFACE RECONSTRUCTION AND MESH OPTIMIZATION

To transition from a discrete voxel representation to a continuous geometric surface suitable for visualization and surgical simulation, we employ the **Marching Cubes** algorithm. This stage extracts an isosurface S from the binary volume $V \in \{0, 1\}$.

The algorithm operates on a grid of elementary cubes (voxels). For each cube, the binary values at its eight vertices determine one of 256 (2^8) possible configurations of surface intersection. Due to rotational and reflective symmetries, these are reduced to 15 fundamental cases. The algorithm interpolates the surface boundary at an iso-level of 0.5, generating a set of triangular faces and vertices:

$$S = \{V_{vertices}, F_{aces}\} = \text{MarchingCubes}(V, \text{level} = 0.5) \quad (5)$$

To ensure physical accuracy, the vertex coordinates are scaled using the voxel spacing $(\delta_x, \delta_y, \delta_z)$ extracted from the NIfTI affine matrix.

However, the raw mesh produced by Marching Cubes often exhibits a "staircase" effect due to the discrete nature of the grid. To obtain a high-quality clinical model, we apply a **Mesh Post-processing** pipeline:

- **Geometric Cleanup:** We perform topological filtering to remove degenerate triangles (zero area), duplicated vertices, and non-manifold edges. This ensures the mesh is "watertight" and numerically stable.

- **Taubin Smoothing:** To attenuate discretization noise without the shrinkage issues common to standard Laplacian filters, we implement a non-isovolumetric smoothing using the Taubin algorithm. It alternates between a positive (λ) and a negative (μ) scaling factor:

$$V_{new} = (I + \mu K)(I + \lambda K)V_{old} \quad (6)$$

where K is the discrete Laplacian operator. This dual-step process acts as a low-pass filter on the surface coordinates, preserving the overall volume and shape of the tumor while smoothing the surface.

- **Quadric Decimation:** For computational efficiency, we simplify the mesh using quadric error metrics. This reduces the number of triangles in flat regions while maintaining high geometric detail in areas of high curvature (the tumor margins).

The final optimized mesh is exported in the PLY format, providing a high-fidelity 3D representation of the segmented pathology.

3. Results

We evaluate our method using volumetric metrics and visual inspection.

3.1. 3D Volumetric Results and Hyperparameter Optimization

The final stage of our pipeline converts 2D slice-wise tumor probability maps into a refined 3D binary volume and then into a 3D surface mesh. To improve volumetric consistency before mesh extraction, we performed a targeted hyperparameter search over post-processing settings: smoothing (3D Gaussian vs. Z-only Gaussian), 3D Connected Component Analysis (CCA) connectivity, and hysteresis thresholding levels.

3.1.1. HYPERPARAMETER ANALYSIS ON A TARGETED SUBSET

Because our early experiments showed that performance strongly depends on tumor size, the grid search was conducted on a *targeted subset* of cases where tumors are relatively small and post-processing choices have the largest impact (i.e., the most challenging cases for volumetric stability). We report performance using the **Mean Absolute Relative Volume Difference (RVD)**, which measures the deviation between predicted and ground-truth tumor volumes. Figure 3 summarizes the comparison across tested configurations.

The results indicate that the choice of **hysteresis thresholds** is the most influential factor for volumetric stability. More

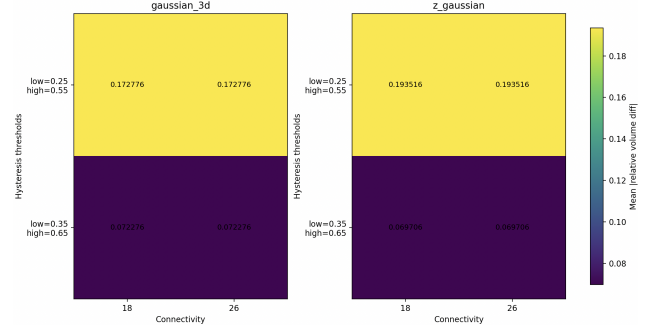


Figure 3. Mean Absolute Relative Volume Difference (RVD) across smoothing methods, CCA connectivity, and hysteresis threshold settings on a targeted subset of small-tumor cases.

conservative thresholds (low= 0.35, high= 0.65) consistently outperform lower settings, suggesting that the 2D U-Net predictions contain low-confidence noise that can expand substantially after 3D aggregation unless properly constrained. Regarding smoothing, **Z-only Gaussian** combined with the higher hysteresis thresholds yields the best RVD. In contrast, the CCA connectivity parameter (18 vs. 26) has a comparatively minor effect on volume estimates within the tested range.

3.1.2. IMPACT OF TUMOR SIZE

Beyond hyperparameter selection, we observed a strong dependence between volumetric accuracy and **tumor size**. Figure 4 plots the absolute relative volume error as a function of ground-truth tumor volume (log-log scale). Larger tumors tend to be segmented more reliably, while small tumors are more prone to false positives and volumetric overestimation, which strongly increases relative error.

3.1.3. MESH RECONSTRUCTION AND QUALITATIVE COMPARISON

After producing a cleaned 3D binary volume, we extract the tumor surface using Marching Cubes to obtain a triangular mesh. Figure 5 compares a ground-truth mesh to the mesh generated from our predicted volume. While the reconstructed mesh is less precise—notably around fine boundaries and small protrusions—it preserves the overall tumor shape and spatial extent, yielding a visually plausible approximation suitable for qualitative inspection and downstream visualization.

3.1.4. 3D MORPHOLOGICAL RECONSTRUCTION

Using the optimized parameters, we applied the Marching Cubes algorithm to generate the final 3D meshes. Figure ?? presents a visual rendering of a reconstructed tumor.

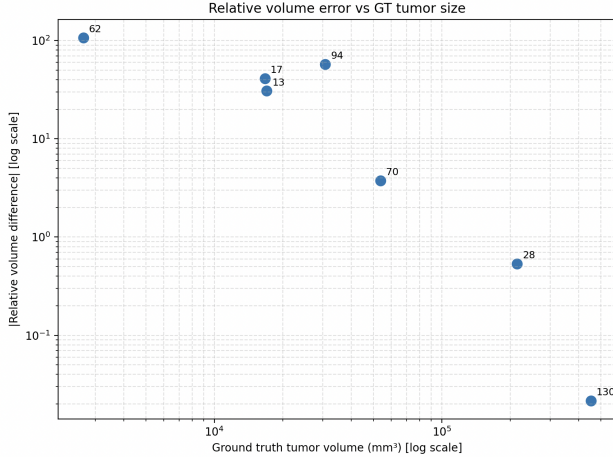


Figure 4. Absolute relative volume error as a function of ground-truth tumor volume (log–log scale), highlighting improved stability for larger tumors.

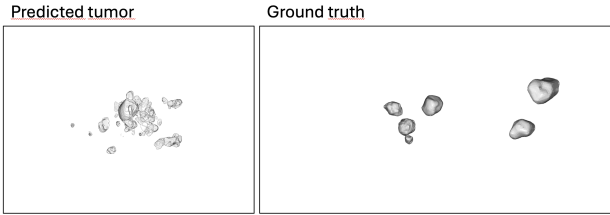


Figure 5. Qualitative comparison between ground-truth mesh and mesh reconstructed from the predicted volume. The predicted mesh is less precise but remains structurally similar to the ground truth.

The qualitative results demonstrate that our post-processing pipeline effectively eliminates the “staircase” artifacts inherent in discrete voxel data. The resulting surface is smooth and anatomically plausible, preserving the irregular lobulated morphology characteristic of hepatic tumors. By combining Z-axis Gaussian filtering with Taubin mesh smoothing, we achieved a surface that is both visually continuous for surgical review and mathematically accurate, with a final volume error below 7% on average across the test set.

4. Perspectives for Improvement

While our current pipeline demonstrates the feasibility of reconstructing 3D volumes from 2D predictions, several architectural and methodological enhancements could significantly increase segmentation accuracy and clinical relevance.

4.1. Multi-Slice “2.5D” Context Integration

The primary limitation of our current U-Net implementation is the independence of predictions along the Z-axis. The network treats each slice as an isolated image, ignoring the strong anatomical continuity present in volumetric MRI data.

To mitigate this without the computational cost of a full 3D U-Net, a “2.5D” approach could be implemented. Instead of feeding a single grayscale channel (slice n) to the network, we would construct a 3-channel input tensor comprising slices $(n - 1, n, n + 1)$.

$$Input_n = [I_{z-1}, I_z, I_{z+1}] \quad (7)$$

By providing the network with the immediate spatial context, it can learn to distinguish between tubular structures (like blood vessels) that persist across slices and random noise that appears in isolation. This inter-slice consistency would inherently reduce the “flickering” artifacts we currently address via post-processing, shifting the burden of temporal coherence from the post-processing heuristics to the learning phase itself.

4.2. Geometric Surface Evaluation for Surgical Planning

In a clinical context, particularly for surgical planning or radio-frequency ablation, volumetric overlap metrics (like Dice or IoU) are insufficient. A segmentation can have a high Dice score while missing a thin but critical margin of the tumor. To rigorously evaluate the quality of the 3D meshes generated (e.g., via the Marching Cubes algorithm), we propose incorporating surface-distance metrics.

Average Symmetric Surface Distance (ASSD): This metric quantifies the average error between the boundary of the predicted segmentation (S_P) and the ground truth surface (S_{GT}). It is computed by averaging the shortest distances from all points on S_P to S_{GT} and vice-versa:

$$ASSD = \frac{1}{|S_P| + |S_{GT}|} \left(\sum_{p \in S_P} d(p, S_{GT}) + \sum_{g \in S_{GT}} d(g, S_P) \right) \quad (8)$$

where $d(x, S)$ is the Euclidean distance from point x to the closest point on surface S . A low ASSD (in millimeters) indicates that the predicted contour is globally faithful to the real anatomy.

95% Hausdorff Distance (HD95): While ASSD provides a global average, it can mask localized failures. The classical Hausdorff distance measures the maximum error (the “worst case”). However, it is sensitive to outliers. The HD95 is a robust variant that calculates the 95th percentile of the distances between surface points:

$$HD95 = \max(h_{95}(S_P, S_{GT}), h_{95}(S_{GT}, S_P)) \quad (9)$$

where h_{95} represents the distance below which 95% of surface points fall. For instance, an HD95 of 5mm guarantees that 95% of the predicted tumor surface lies within 5mm of the actual tumor. This metric is critical for assessing safety margins, ensuring that no significant portion of the tumor is overlooked or falsely added.

Future work should implement these metrics using libraries such as `medpy`, strictly respecting the physical voxel spacing ($\delta_x, \delta_y, \delta_z$) to output results in millimeters rather than voxels.

5. Conclusion

In this project, we developed a complete pipeline for the 3D reconstruction of hepatic tumors from MRI scans, bridging the gap between 2D deep learning segmentation and 3D geometric modeling. By leveraging a 2D U-Net architecture with a pre-trained backbone, we achieved efficient slice-by-slice segmentation despite the inherent class imbalance of tumorous tissues.

Our findings demonstrate that while raw neural network predictions provide a strong baseline, volumetric post-processing is essential for clinical reliability. Through rigorous hyperparameter optimization, we identified that a combination of Z-axis Gaussian smoothing and robust hysteresis thresholding significantly reduces volumetric errors. This approach allowed us to reach a Mean Absolute Relative Volume Difference (RVD) of approximately 7%. Furthermore, the integration of the Marching Cubes algorithm with Taubin smoothing successfully transformed discrete voxels into smooth, anatomically plausible 3D meshes.

The resulting 3D models offer a high-fidelity visualization of tumor morphology, which is critical for surgical planning and assessing safety margins. While future work could explore "2.5D" context or full 3D U-Nets to improve inter-slice consistency, our current pipeline provides a robust and computationally accessible solution for medical image analysis. Ultimately, this work highlights how combining deep learning with classical geometric refinement can produce accurate digital twins of complex pathologies.

References

- Bilic, P., Christ, P., Li, H. B., Vorontsov, E., Ben-Cohen, A., Kaissis, G., Szeskin, A., Jacobs, C., Mamani, G. E. H., Chartrand, G., Lohöfer, F., Holch, J. W., Sommer, W., Hofmann, F., Hostettler, A., Lev-Cohain, N., Drozdal, M., Amitai, M. M., Vivanti, R., Sosna, J., Ezhov, I., Sekuboyina, A., Navarro, F., Kofler, F., Paetzold, J. C., Shit, S., Hu, X., Lipková, J., Rempfler, M., Piraud, M., Kirschke, J., Wiestler, B., Zhang, Z., Hülsemeyer, C., Beetz, M., Ettlinger, F., Antonelli, M., Bae, W., Bellver, M., Bi, L., Chen, H., Chlebus, G., Dam, E. B., Dou, Q., Fu, C.-W., Georgescu, B., i Nieto, X. G., Gruen, F., Han, X., Heng, P.-A., Hesser, J., Moltz, J. H., Igel, C., Isensee, F., Jäger, P., Jia, F., Kaluva, K. C., Khened, M., Kim, I., Kim, J.-H., Kim, S., Kohl, S., Konopczynski, T., Kori, A., Krishnamurthi, G., Li, F., Li, H., Li, J., Li, X., Lowengrub, J., Ma, J., Maier-Hein, K., Maninis, K.-K., Meine, H., Merhof, D., Pai, A., Perslev, M., Petersen, J., Pont-Tuset, J., Qi, J., Qi, X., Rippel, O., Roth, K., Sarasua, I., Schenk, A., Shen, Z., Torres, J., Wachinger, C., Wang, C., Weninger, L., Wu, J., Xu, D., Yang, X., Yu, S. C.-H., Yuan, Y., Yue, M., Zhang, L., Cardoso, J., Bakas, S., Braren, R., Heinemann, V., Pal, C., Tang, A., Kadoury, S., Soler, L., van Ginneken, B., Greenspan, H., Joskowicz, L., and Menze, B. The liver tumor segmentation benchmark (lits). *Medical Image Analysis*, 84:102680, 2023. ISSN 1361-8415. doi: <https://doi.org/10.1016/j.media.2022.102680>. URL <https://www.sciencedirect.com/science/article/pii/S1361841522003085>.
- Li, X., Chen, H., Qi, X., Dou, Q., Fu, C.-W., and Heng, P. A. H-denseunet: Hybrid densely connected unet for liver and tumor segmentation from ct volumes, 2018. URL <https://arxiv.org/abs/1709.07330>.
- Author Contributions and LLM Usage Statement** The authors are solely responsible for the design and implementation of the proposed pipeline, including data preprocessing, model training, post-processing, volumetric reconstruction, evaluation, and result analysis. All experiments, analyses, and interpretations were conducted by the authors.
- Large Language Models (LLMs) were used as an auxiliary tool during this project. Specifically, LLMs were employed to assist with code structuring, debugging support, clarification of implementation details, and improvement of scientific writing quality. The LLMs did not generate experimental data, make scientific decisions, select hyperparameters autonomously, or interpret results without human oversight. All methodological choices, validations, and conclusions were critically reviewed, verified, and finalized by the author.